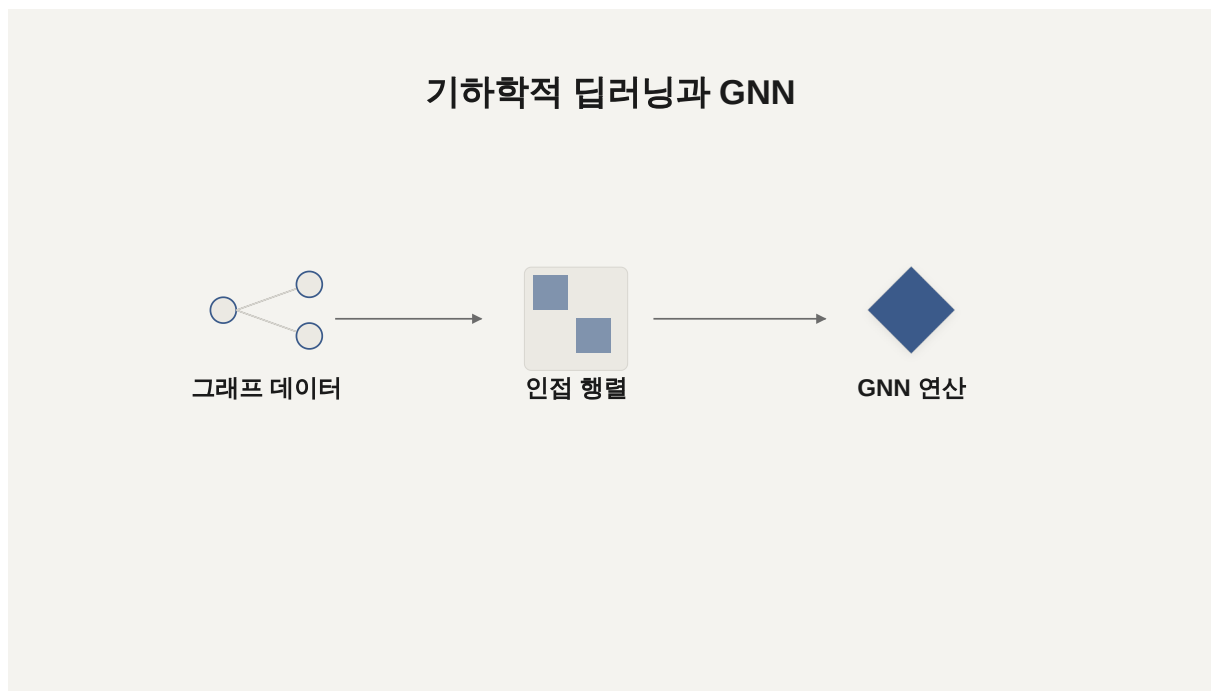


STEP 5. PyTorch Geometric(PyG) 실습 (2~3주)

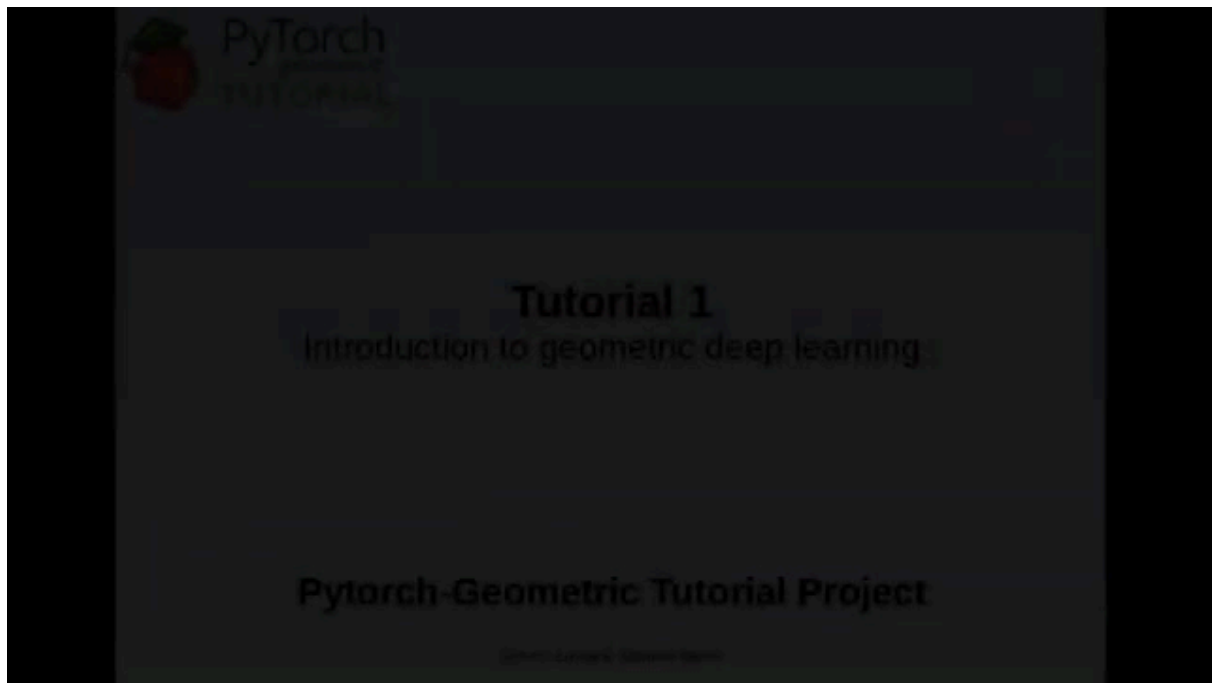
기존 딥러닝 모델이 그래프 데이터에 적용되기 어려운 이유는 무엇일까? 그래프의 **노드 순서 불변성**과 **가변적인 크기** 때문에 일반적인 행렬 기반 모델로는 학습이 불가능하며, 이를 해결하기 위해 그래프 신경망(GNN)이 등장했습니다.

1. Pytorch 기하학 튜토리얼: Pytorch 기하학 소개

기존 딥러닝 모델이 그래프 데이터에 적용되기 어려운 이유와 이를 해결하기 위한 그래프 신경망(GNN)의 필요성을 소개하고, Pytorch Geometric 프레임워크를 활용한 기하학적 딥러닝 학습 여정을 시작한다.



1.1. 기하학적 딥러닝의 필요성 및 동기



1. 기존 딥러닝의 한계점

- a. 이미지, 음성 등 정형화된 데이터에 비해 그래프 데이터는 기존 딥러닝 모델 적용에 어려움이 있다.
- b. 그래프 데이터는 **노드 순서 불변성**과 **가변적인 크기**라는 특징 때문에 일반적인 행렬 기반 모델로는 학습이 불가능하다.

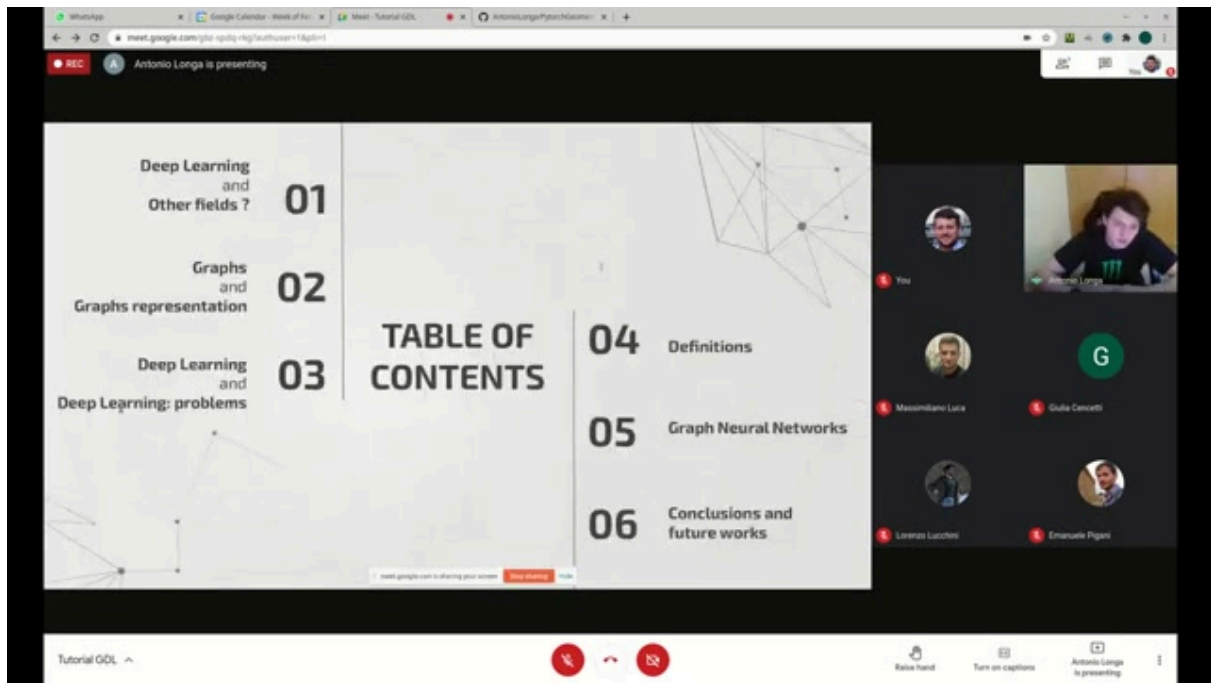
2. 그래프 신경망(GNN)의 등장 배경

- a. 그래프 데이터의 특성을 해결하기 위해 그래프 신경망(GNN)이 등장했다.

3. 튜토리얼 프로젝트 소개

- a. 본 튜토리얼은 Pytorch Geometric 프레임워크를 사용하여 기하학적 딥러닝을 학습하는 여정이다.
- b. 매주 토요일 정기적인 모임을 통해 기하학적 딥러닝에 대해 논의하고 관련 내용을 발표한다.
- c. 누구나 참여 가능하며, 오픈 소스 프로젝트로 코드를 활용하고 기여할 수 있다.
- d. 공식 웹사이트(geometric deep learning tutorial)에서 강의 요약, 영상 링크, GitHub 저장소를 제공한다.

1.2. 그래프 데이터의 이해와 기존 딥러닝 적용의 어려움



1. 그래프의 정의 및 종류

- a. 그래프는 정점(노드)과 간선(엣지)의 쌍으로 정의된다.
- b. 방향 그래프, 노드 레이블 그래프, 무방향 그래프, 엣지 레이블 그래프 등 다양한 종류가 있다.

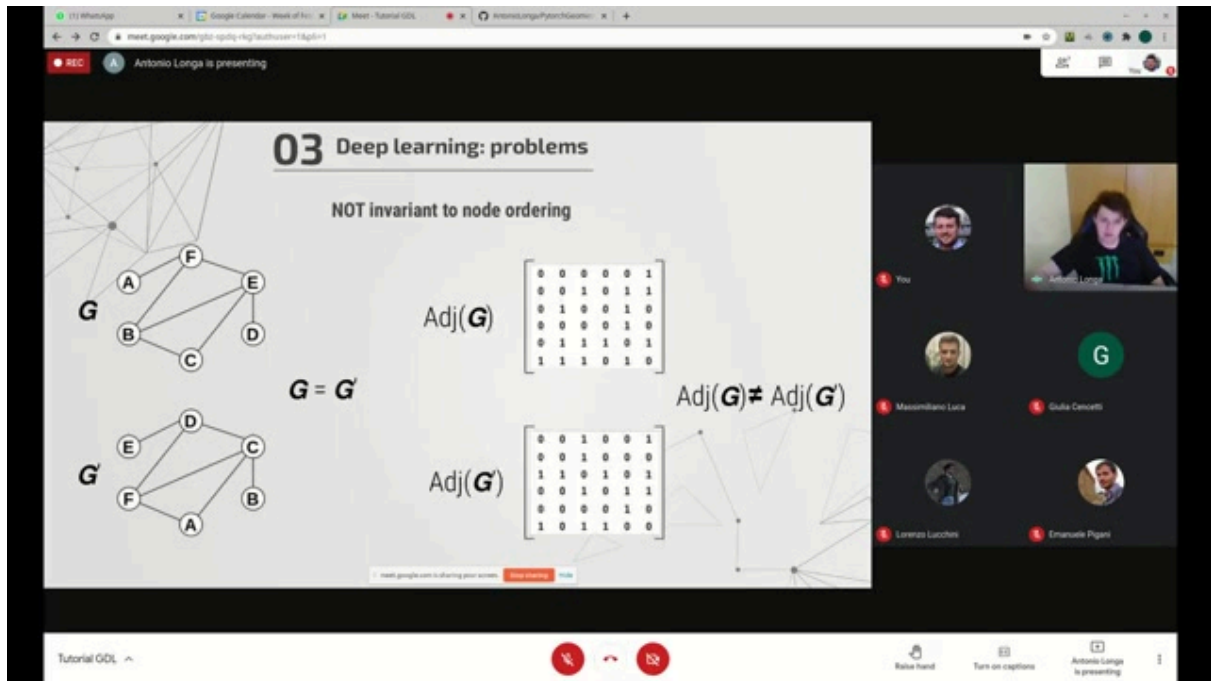
2. 그래프 표현 방식

- a. 그래프는 **인접 행렬(Adjacency Matrix)**로 표현될 수 있다.
- b. 인접 행렬은 그래프의 노드 수만큼의 크기를 가지는 정방 행렬이며, 두 노드 간 연결이 있으면 1, 없으면 0으로 표시된다.
- c. 무방향 그래프의 경우 인접 행렬은 대칭적이다.

3. 기존 딥러닝 모델 적용의 문제점

- a. **가변적인 크기 문제**: 새로운 노드가 추가되면 인접 행렬의 크기가 커져 기존 학습 모델에 입력으로 사용할 수 없다.
- b. **노드 순서 불변성 문제**: 그래프의 노드 순서가 바뀌어도 그래프 자체는 동일하지만, 인접 행렬은 달라진다.
 - i. 예를 들어, 두 개의 동일한 그래프라도 노드에 부여된 이름(순서)에 따라 인접 행렬이 완전히 달라질 수 있다.
- c. 이러한 문제 때문에 이미지(픽셀 그리드)나 음성(정보 체인)과 같은 정형화된 데이터와 달리 그래프 데이터는 기존 딥러닝 모델, 특히 컨볼루션 신경망을 직접 적용하기 어렵다.

1.3. 그래프 신경망(GNN)의 기본 개념



1. 그래프 데이터의 추가 정보

- 그래프는 인접 행렬 외에도 각 노드의 특징을 나타내는 **특징 벡터(Feature Vector)** 또는 **특징 행렬(Feature Matrix)**을 가질 수 있다.
- 특징 행렬의 행 수는 노드 수와 같고, 열 수는 각 노드의 특징 차원과 같다.
- G는 그래프, A는 인접 행렬, X는 노드 특징 행렬을 나타내는 일반적인 표기법이다.

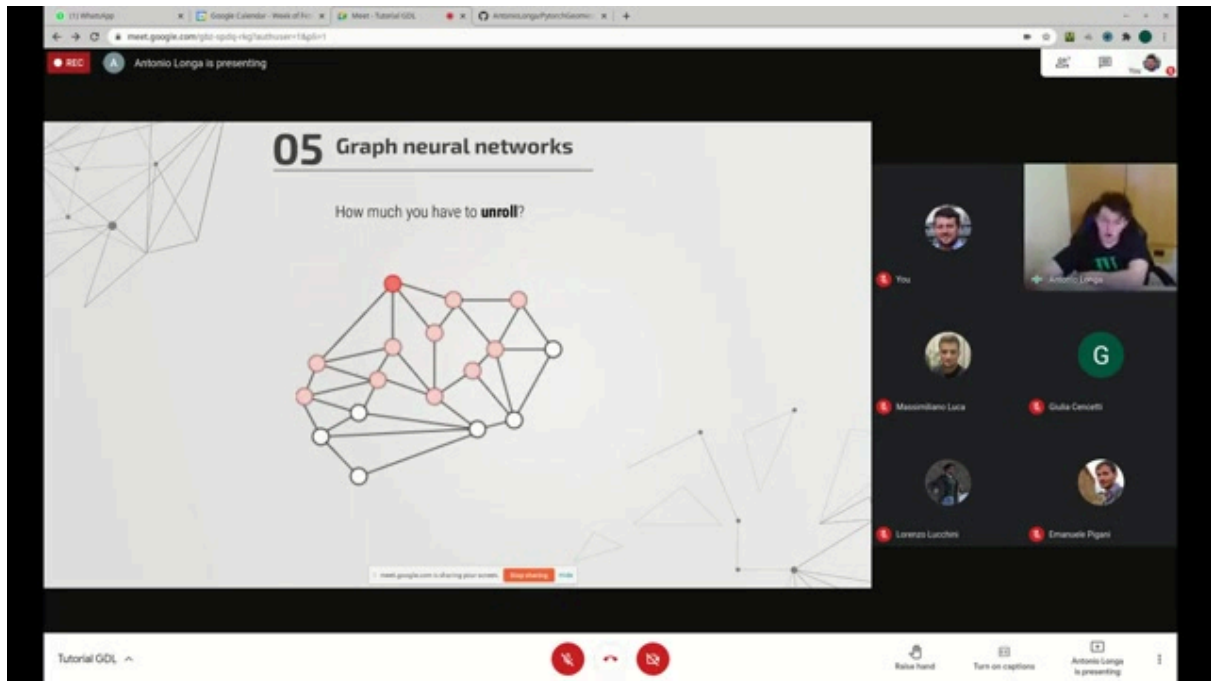
2. 계산 그래프(Computation Graph)

- GNN은 각 노드에 대한 **계산 그래프**를 정의하여 정보를 추론한다.
- 계산 그래프는 특정 노드와 그 이웃 노드들의 정보를 포함하는 구조이다.
- 예를 들어, 노드 A에 대한 계산 그래프는 노드 A 자체와 A에 연결된 노드(B, E)의 특징 정보를 포함한다.
- 이 정보들은 신경망(박스)을 통해 처리되며, 이웃 노드들의 특징 정보와 결합된다.
- 순서 불변 집계(Order Invariant Aggregation)**가 중요하다.
 - 합계, 평균과 같이 노드의 순서에 영향을 받지 않는 집계 함수를 사용해야 한다.
- 각 노드는 자신만의 계산 그래프를 가지며, 이웃 노드들의 정보가 반복적으로 포함될 수 있다.

3. 계산 그래프의 깊이(Unrolling)

- a. 계산 그래프를 얼마나 깊게 펼칠지(unroll) 결정해야 한다.
- b. 이는 추론하려는 노드와 관련된 정보의 범위를 결정한다.
- c. 소셜 네트워크 예시처럼, 너무 멀리 떨어진 노드의 정보는 관련성이 낮을 수 있다.
- d. 네트워크의 지름(diameter)을 고려하여 불필요하게 넓은 범위의 정보를 수집하지 않도록 해야 한다.

1.4. 그래프 신경망의 수학적 표현 및 학습



1. 노드 임베딩(Node Embedding)

- a. 레이어 0에서의 노드 V 의 표현(representation)은 노드 V 자체의 특징 벡터이다.
- b. 레이어 $K+1$ 에서의 노드 V 의 임베딩(embedding)은 이전 레이어의 정보와 이웃 노드들의 정보를 종합하여 계산된다.
- c. 최종적으로 노드 V 의 임베딩 $Z(V)$ 는 마지막 레이어의 표현이다.

2. 레이어별 임베딩 계산

- a. 레이어 $K+1$ 에서의 노드 V 의 임베딩은 이전 레이어에서의 노드 V 의 임베딩과 이웃 노드들의 임베딩을 집계하여 계산된다.
- b. 집계는 이웃 노드들의 이전 레이어 임베딩의 **평균**을 사용한다.
- c. 여기에 학습 가능한 가중치 행렬 W 와 편향 B 가 적용되고, 비선형 활성화 함수를 거친다.

- d. 즉, $h_v^{(k+1)} = \sigma(W^{(k)} \cdot \frac{1}{|N(v)|} \sum_{u \in N(v)} h_u^{(k)} + B^{(k)} * h_v^{(k)})$ 와 같은 형태이다.
- e. 여기서 $h_v^{(k)}$ 는 레이어 k에서의 노드 v의 임베딩, $N(v)$ 는 노드 v의 이웃 집합이다.
- f. 각 레이어의 파라미터(W, B)는 모든 계산 그래프에서 공유되어, 새로운 노드나 학습되지 않은 노드에 대한 추론이 가능하다.

3. GraphSAGE 모델

- a. GraphSAGE는 기존 GNN 모델의 변형으로, 특히 집계 함수와 정보 결합 방식에 변화를 주었다.
- b. **일반화된 집계 함수 사용**: 평균 대신 다양한 집계 함수(폴링 전략: 평균, 최대값 등)를 사용할 수 있다.
- c. **정보 결합 방식 변경**: 이전 레이어의 노드 자체 정보와 이웃 노드들의 집계 정보를 합치는 대신 **연결(concatenate)**한다.
- d. 집계 함수로는 폴링(평균, 최대값 등)이나 LSTM 등을 사용할 수 있다.
- e. LSTM은 순서 불변성이 아니지만, 무작위 순열을 이용한 반복 계산으로 이를 극복하는 연구도 있다.

1.5. Pytorch Geometric을 이용한 실습

1. 데이터셋 로드 및 이해

- a. Pytorch Geometric 라이브러리를 사용하여 미리 정의된 데이터셋(예: Cora)을 다운로드하고 로드한다.
- b. 데이터셋은 다운로드 후 로컬에 저장되어 재사용 시 다운로드 과정을 생략한다.
- c. **Cora 데이터셋**: 논문 인용 관계를 나타내는 그래프 데이터셋이다.
- d. 데이터셋의 주요 속성:
 - i. 이름: Cora
 - ii. 그래프 개수: 1개
 - iii. 클래스 개수: 7개
 - iv. 노드 특징 차원: 1433개
 - v. 엣지 특징: 없음
- e. **그래프 표현**:

- i. 엣지 인덱스(Edge Index): 노드 간 연결 정보를 담고 있으며, (2, num_edges) 형태의 텐서이다.
 - ii. 노드 특징(X): 각 노드의 1433차원 특징 벡터로 구성된 행렬이다. (num_nodes, num_node_features) 형태이다.
 - iii. 노드 레이블(Y): 각 노드가 속한 클래스 정보를 나타낸다.
- f. **학습/검증/테스트 분할:**
- i. 학습 마스크(Training Mask), 검증 마스크(Validation Mask), 테스트 마스크(Test Mask)를 통해 노드를 분할한다.

2. 간단한 GNN 모델 구현 및 학습

- a. 필요한 라이브러리(torch, torch_geometric, SageConv)를 임포트한다.
- b. **SageConv** 레이어를 사용하여 신경망 모델을 정의한다.
 - i. 입력 채널은 노드 특징 차원(1433), 출력 채널은 클래스 개수(7)로 설정한다.
 - ii. 집계 함수로 'max'를 사용하도록 지정한다.
- c. 모델의 **forward** 함수는 입력 데이터(x)와 엣지 인덱스를 받아 **SageConv** 레이어를 통과시킨 후, 로그 소프트맥스(log_softmax)를 반환한다.
- d. 모델과 데이터를 CPU 또는 GPU로 이동시킨다.
- e. Adam 옵티마이저를 사용하여 모델을 학습시킨다.
- f. 학습 및 테스트 함수를 정의하고, 100 에폭 동안 모델을 학습시킨다.
- g. **결과:** 학습 결과, 검증 및 테스트 정확도가 크게 향상되지 않고 안정적인 상태를 보인다.
 - i. 이는 드롭아웃, 선형 레이어 등 추가적인 기법이 없는 매우 단순한 모델이기 때문이다.

3. 질의응답 및 향후 계획

- a. **다양한 문제에 대한 적용:** GNN은 노드 분류 외에도 회귀, 엣지 분류 등 다양한 문제에 적용 가능하다.
 - i. 노드 임베딩은 각 노드의 학습된 표현으로, 이를 활용하여 다양한 예측 작업을 수행할 수 있다.
 - ii. 엣지 관련 작업은 엣지 쌍을 중심으로 계산 그래프를 구축하여 수행할 수 있다.
- b. **Self-loop 처리:** Self-loop는 노드 자체의 정보를 반복적으로 입력하는 효과를 줄 수 있으며, 이를 제한하거나 다른 방식으로 처리하는 방법이 연구될 수 있다.

- c. **계산 그래프 깊이와 네트워크 깊이:** 계산 그래프의 깊이는 이웃 노드 정보를 얼마나 확장하여 수집할지를 결정하며, 네트워크의 깊이는 모델의 복잡성을 결정한다.
 - i. 너무 깊은 계산 그래프는 전체 그래프를 고려하게 되어 비효율적일 수 있다.
- d. **향후 학습 계획:** 다음 튜토리얼에서는 Pytorch 기본 및 Pytorch Geometric의 상세한 구현 방법을 다룰 예정이다.